

# Jane Street 시장 예측: 모델 기반 분석 전략 탐구

Kaggle 1팀 최은서, 김가현, 박연수, 박주연, 최인재



# Table of Contents

1. Overview
2. Data Preprocessing
3. Feature Selection
4. Train & Test
5. Conclusion

# Jane Street 시장 예측: 모델 기반 분석 전략 탐구

## Kaggle Jane Street 대회란?

세계적인 금융 기업 'Jane Street'가 주최하는 금융 시계열 데이터 예측 대회  
실제 시장 데이터를 기반으로 하나, **모든 피처가 익명으로 처리된** 것이  
가장 큰 특징  
참가자는 익명 데이터를 분석하여 미래의 특정 시장 반응을 예측하는 모델을 만들어야 함

## 프로젝트 참여 목적

현재 이 대회는 종료되어 실제 순위 경쟁은 불가능  
따라서 본 양질의 데이터를 활용하여,  
데이터 분석 및 머신러닝 모델링 기법을 **학습하고 적용**하는 것을  
목표로 함



# Jane Street 시장 예측: 모델 기반 분석 전략 탐구

## 제공된 데이터 구성

### train.parquet

약 500일간, 240만 개 이상의 거래 기록을 담은 **훈련 데이터**

### test.parquet

모델의 성능을 측정하기 위한 **평가 데이터**

### features.csv

79개 Feature가 어떤 그룹에 속하는지 알려주는 메타데이터

### lags.parquet

이전 날짜(date\_id - 1)의 responder 값들을 lag 형태로 제공

### responders.csv

9개 responder 변수의 메타데이터

### kaggle\_evaluation

Kaggle에서 제공하는 실시간 평가 **API 코드**

#### 데이터 구성

##### COMPETITIONS

- ▼  Jane Street Real-Time Market Data  
- ▶  kaggle\_evaluation
- ▶  lags.parquet
- ▶  test.parquet
- ▶  train.parquet
-  features.csv
-  responders.csv
-  sample\_submission.csv

#### train.parquet

예시

date\_id

time\_id

symbol\_id

weight

feature\_00

...

responder\_4

responder\_5

responder\_6

responder\_7

responder\_8

# Jane Street 시장 예측: 모델 기반 분석 전략 탐구

## 데이터의 주요 특성

### 익명 피처

feature\_0부터 feature\_78까지, 의미를 알 수 없는 **79개의 숫자형 피처**로 구성되어 있어서 데이터 자체의 통계적 패턴 분석에 집중해야 함

### 고차원 & 대용량 (High-Dimensional & Large-Scale)

79개로 다수인 Feature를 동시에 고려해야 하며, 데이터가 방대하므로 효율적인 처리 방법이 중요

### 결측치 존재

데이터 내에 결측치가 상당량 존재하므로 이를 어떻게 처리할지에 대한 전략이 필요

| date_id | time_id | symbol_id | weight   | feature_00 | feature_01 | feature_02 | feature_03 | feature_04 | feature_05 | feature_06 | feature_07 | feature_08 | feature_09 | feature_10 | feature_11 | feature_12 |
|---------|---------|-----------|----------|------------|------------|------------|------------|------------|------------|------------|------------|------------|------------|------------|------------|------------|
| i16     | i16     | i8        | f32      | f32        | f32        | f32        | f32        | f32        | f32        | f32        | f32        | f32        | i8         | i8         | i16        | f32        |
| 0       | 0       | 1         | 3.889038 | null       | null       | null       | null       | null       | 0.851033   | 0.242971   | 0.2634     | -0.891687  | 11         | 7          | 76         | -0.883028  |
| 0       | 0       | 7         | 1.370613 | null       | null       | null       | null       | null       | 0.676961   | 0.151984   | 0.192465   | -0.521729  | 11         | 7          | 76         | -0.865307  |
| 0       | 0       | 9         | 2.285698 | null       | null       | null       | null       | null       | 1.056285   | 0.187227   | 0.249901   | -0.77305   | 11         | 7          | 76         | -0.675719  |
| 0       | 0       | 10        | 0.690606 | null       | null       | null       | null       | null       | 1.139366   | 0.273328   | 0.306549   | -1.262223  | 42         | 5          | 150        | -0.694008  |
| 0       | 0       | 14        | 0.44057  | null       | null       | null       | null       | null       | 0.9552     | 0.262404   | 0.344457   | -0.613813  | 44         | 3          | 16         | -0.947351  |

# Jane Street 시장 예측: 모델 기반 분석 전략 탐구

## 예측 대상

예측해야 할 최종 목표는 **responder\_6** 라는 숫자 값  
-> 특정 거래에 대한 '단기 시장 반응'을 나타냄.

공식적으로는 공개되지 않았지만, Kaggle 토론에서 일부  
참가자들이 **SMA(단순 이동 평균) 20**과 관련있다고 추정함.

## 평가 지표

제출 데이터는 **responder\_6** 예측치의 **R-square** 값으로  
평가함.

### 핵심 과제

#### 실시간 예측 환경의 어려움

이 대회는 데이터가 한 번에 한 줄씩 주어지는 실시간 상황을 가정하므로, 다음과 같은 현실적인 어려움들을 극복해야 함.

##### 1. 지연

제한된 시간 안에 빠르게 예측을 계산해야 함

##### 2. 누적 에러

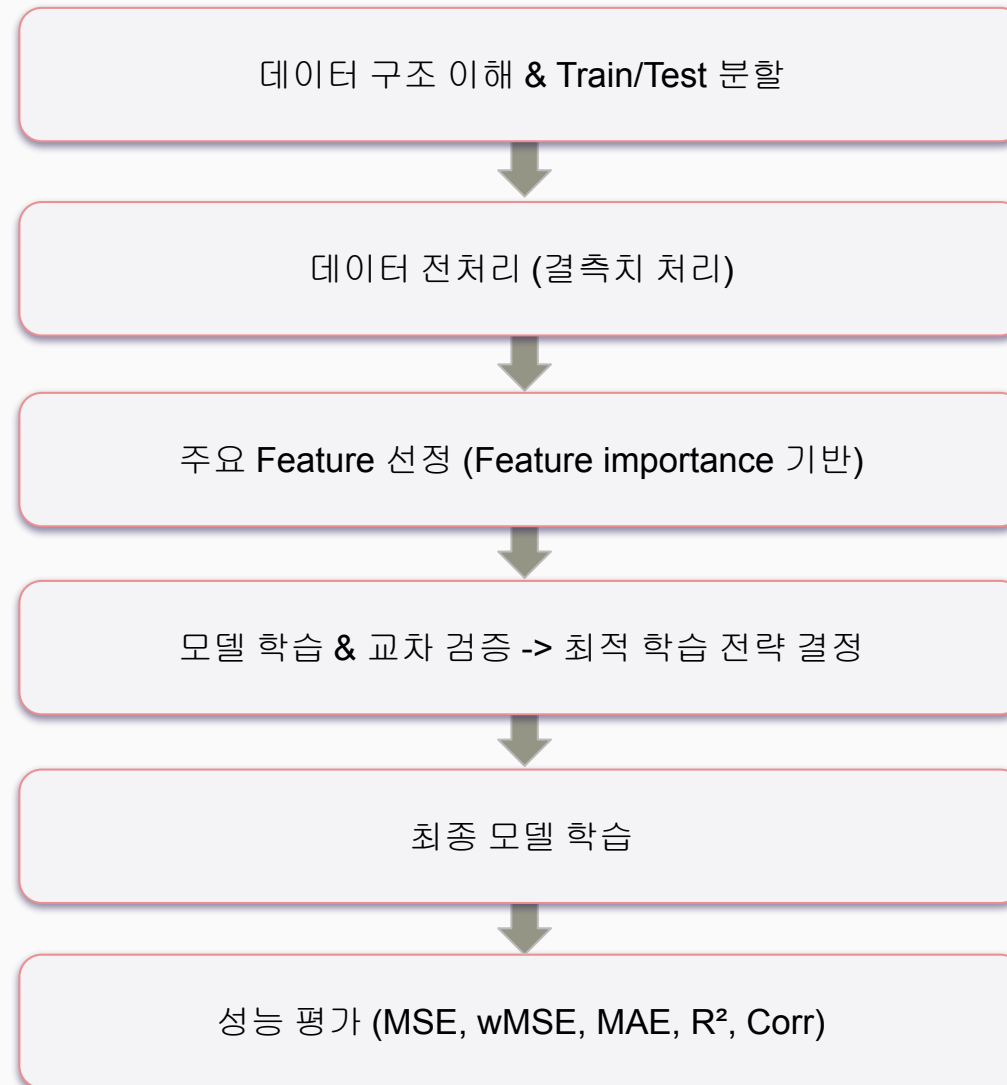
작은 예측 실수가 이후의 예측에 계속해서 영향을 줄 수 있음

##### 3. 제한된 정보

독립변수의 자세한 정보를 모른 채 종속변수를 예측해야 함

# Pipeline

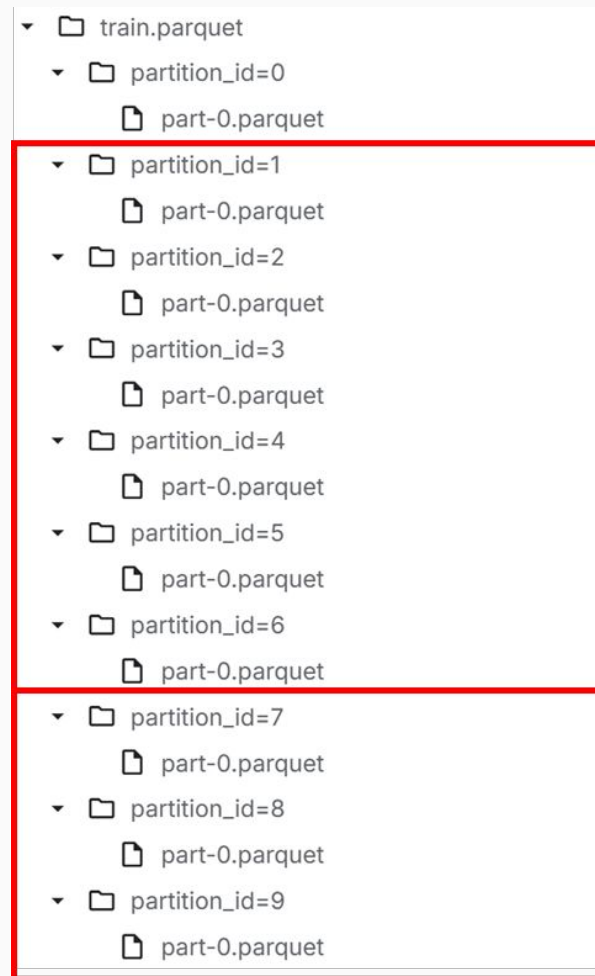
---





# 데이터 분할 및 검증 방식

- 본 캐글 프로젝트는 제출이 불가능했음.
- 파티션 0~6은 Train 데이터로, 파티션 7~9는 Test 데이터로 임의 분할

**Train****Test**



## 결측치 처리

### Partition 0~6 에 대해 missing rate 조사

- Feature 21,26,27,31 ➡ drop
- Feature 00,01,02,03,04 ➡ partition 0,1 에 대해 50% 이상 결측 확인, drop
- 결측률이 0~15% 수준인 피처는 트리 기반 모델이 결측을 자동 처리할 수 있어 제거하지 않고 유지함.

| column              | nulls    | rows     | missing_rate |
|---------------------|----------|----------|--------------|
| str                 | u32      | i32      | f64          |
| "responder_6_lag_1" | 11802106 | 11802114 | 0.999999     |
| "feature_21"        | 8212377  | 11802114 | 0.695839     |
| "feature_26"        | 8212377  | 11802114 | 0.695839     |
| "feature_27"        | 8212377  | 11802114 | 0.695839     |
| "feature_31"        | 8212377  | 11802114 | 0.695839     |
| "feature_00"        | 3182052  | 11802114 | 0.269617     |
| "feature_01"        | 3182052  | 11802114 | 0.269617     |
| "feature_02"        | 3182052  | 11802114 | 0.269617     |
| "feature_03"        | 3182052  | 11802114 | 0.269617     |
| "feature_04"        | 3182052  | 11802114 | 0.269617     |
| "feature_39"        | 1818907  | 11802114 | 0.154117     |
| "feature_42"        | 1818907  | 11802114 | 0.154117     |
| "feature_50"        | 1772574  | 11802114 | 0.150191     |
| "feature_53"        | 1772574  | 11802114 | 0.150191     |
| "feature_41"        | 435922   | 11802114 | 0.036936     |
| "feature_44"        | 435922   | 11802114 | 0.036936     |
| "feature_52"        | 388024   | 11802114 | 0.032877     |
| "feature_55"        | 388024   | 11802114 | 0.032877     |
| "feature_15"        | 333462   | 11802114 | 0.028254     |
| "feature_45"        | 306677   | 11802114 | 0.025985     |
| "feature_46"        | 306677   | 11802114 | 0.025985     |
| "feature_65"        | 306677   | 11802114 | 0.025985     |
| "feature_66"        | 306677   | 11802114 | 0.025985     |
| "feature_62"        | 288291   | 11802114 | 0.024427     |
| "feature_64"        | 235171   | 11802114 | 0.019926     |
| "feature_63"        | 225417   | 11802114 | 0.0191       |
| "feature_73"        | 131538   | 11802114 | 0.011145     |
| "feature_74"        | 131538   | 11802114 | 0.011145     |
| "feature_32"        | 131527   | 11802114 | 0.011144     |
| "feature_33"        | 131527   | 11802114 | 0.011144     |
| "feature_58"        | 131522   | 11802114 | 0.011144     |
| "feature_40"        | 67559    | 11802114 | 0.005724     |
| "feature_43"        | 67559    | 11802114 | 0.005724     |
| "feature_17"        | 55853    | 11802114 | 0.004732     |
| "feature_08"        | 19527    | 11802114 | 0.001655     |
| "feature_51"        | 13805    | 11802114 | 0.00117      |
| "feature_54"        | 13805    | 11802114 | 0.00117      |
| "feature_37"        | 849      | 11802114 | 0.000072     |
| "feature_75"        | 303      | 11802114 | 0.000026     |
| "feature_76"        | 303      | 11802114 | 0.000026     |
| "feature_16"        | 250      | 11802114 | 0.000021     |
| "feature_18"        | 219      | 11802114 | 0.000019     |
| "feature_19"        | 219      | 11802114 | 0.000019     |
| "feature_56"        | 219      | 11802114 | 0.000019     |
| "feature_57"        | 219      | 11802114 | 0.000019     |
| "feature_47"        | 87       | 11802114 | 0.000007     |

# Feature importance 탐색 과정

- 모델이 학습 과정에서 각 피처가 얼마나 예측에 기여했는지를 수치로 나타낸 것
- 분기(split) 시 손실 감소량(Gain)을 기준으로 산출 -> 값이 클수록 모델 예측에 중요한 역할을 한 피처

```
for pid in range(7): # partition_id=0~6
    print(f"=== Partition {pid} ===")
```

for문 활용 : 메모리 절약

```
# -----
# 3) LightGBM 학습 (partition별)
# -----
model = lgb.LGBMRegressor(
    n_estimators=500,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42,
    n_jobs=-1
)
model.fit(X, y)

booster = model.booster_
imp_df = pd.DataFrame({
    "feature": model.feature_name_,
    "gain": booster.feature_importance(importance_type="gain"),
})
imp_df["gain_norm"] = imp_df["gain"] / (imp_df["gain"].sum() + 1e-12)
imp_df = imp_df.sort_values("gain", ascending=False).reset_index(drop=True)

# -----
# 4) 상위 30개 feature 추출 (리스트만 저장)
TOP_N = 30
high_features = imp_df["feature"].iloc[:TOP_N].tolist()

# 파티션별로 리스트 저장
high_features_dict[f"high_features_{pid}"] = high_features
```

Light GBM 모델 사용

## 버전 1) Partition별 top30의 교집합

- ▶ 모든 파티션에 공통으로 등장한 feature 개수: 20
- ▶ 공통 feature 목록: ['feature\_61', 'feature\_58', 'feature\_60', 'feature\_15', 'feature\_08', 'feature\_24', 'feature\_30', 'feature\_47', 'feature\_07', 'feature\_29', 'feature\_38', 'feature\_25', 'feature\_62', 'feature\_06', 'feature\_22', 'feature\_05', 'feature\_23', 'feature\_20', 'feature\_37', 'feature\_28']

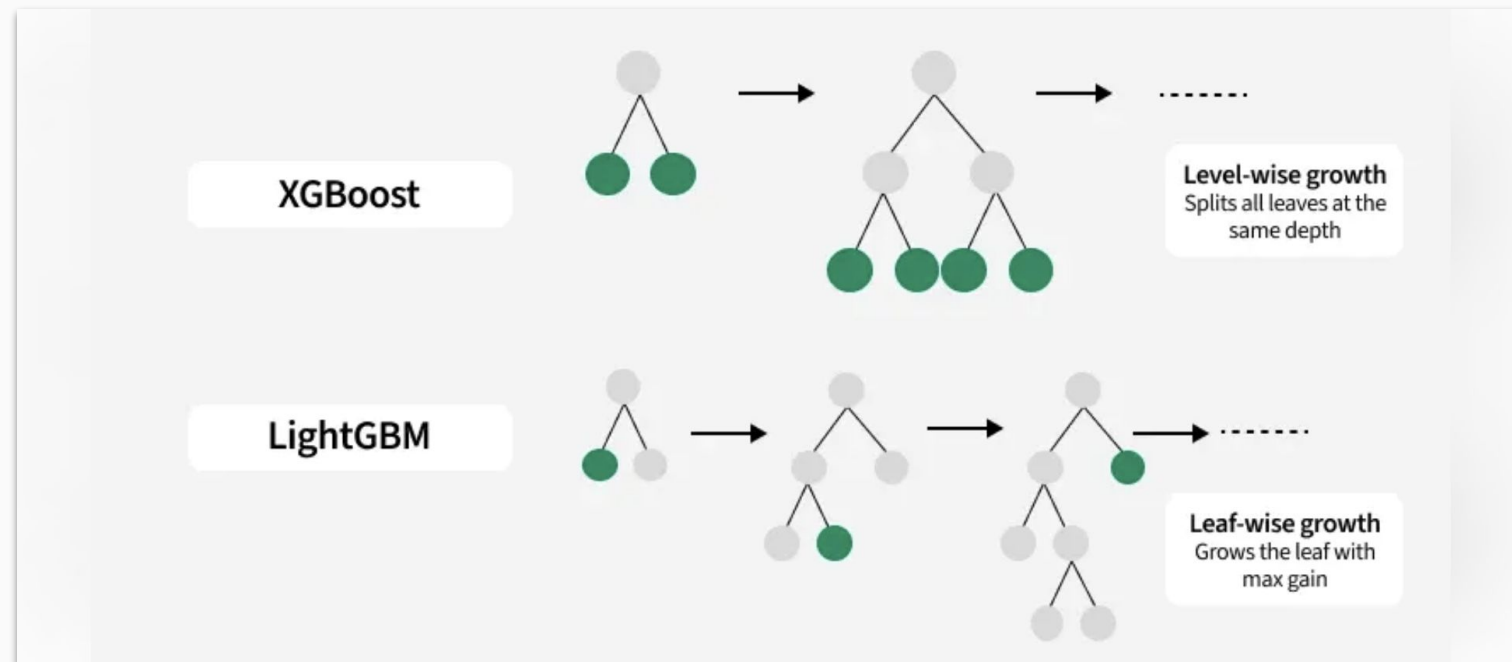
## 버전 2) List 3개 이상 등장하는 피처 집합

- ▶ 3개 이상 리스트에 나타난 feature 개수: 33
- ▶ feature 목록: ['feature\_06', 'feature\_20', 'feature\_22', 'feature\_07', 'feature\_28', 'feature\_25', 'feature\_30', 'feature\_24', 'feature\_29', 'feature\_62', 'feature\_61', 'feature\_23', 'feature\_47', 'feature\_38', 'feature\_15', 'feature\_60', 'feature\_64', 'feature\_58', 'feature\_37', 'feature\_08', 'feature\_59', 'feature\_05', 'feature\_49', 'feature\_69', 'feature\_56', 'feature\_17', 'feature\_33', 'feature\_74', 'feature\_73', 'feature\_36', 'feature\_21', 'feature\_50', 'feature\_72']

# LightGBM 이란?

Leaf-wise growth 방식으로, 손실이 큰 부분을 파고들고 쪼개며 개선시키는 방식

층(Level) 단위로 뺀 XGBoost등과 달리, 더 복잡한 패턴을 빨리 학습하고 오류를 빠르게 해결할 수 있음



| 파라미터명<br>(파이썬 래퍼)                | 파라미터명<br>(사이킷런 래퍼)               | 설명                                                                                                                 |
|----------------------------------|----------------------------------|--------------------------------------------------------------------------------------------------------------------|
| <b>num_iterations</b><br>(100)   | <b>n_estimators</b><br>(100)     | - 반복 수행 트리개수 지정<br>- 너무 크면 과적합 발생                                                                                  |
| <b>learning_rate</b><br>(0.1)    | <b>learning_rate</b><br>(0.1)    | - 학습률                                                                                                              |
| <b>max_depth</b><br>(-1)         | <b>max_depth</b><br>(-1)         | - 최대 깊이<br>- default → 깊이에 제한이 없음                                                                                  |
| <b>min_data_in_leaf</b><br>(20)  | <b>min_child_samples</b><br>(20) | - 최종 리프 노드가 되기 위한 레코드수<br>- 과적합 제어용                                                                                |
| <b>num_leaves</b><br>(31)        | <b>num_leaves</b><br>(31)        | - 하나의 트리가 가지는 최대 리프 개수                                                                                             |
| <b>boosting</b><br>(‘gbdt’)      | <b>boosting_type</b><br>(‘gbdt’) | - gbdt : 일반적인 그래디언트부스팅 트리<br>- rf : 랜덤포레스트                                                                         |
| <b>bagging_fraction</b><br>(1.0) | <b>subsample</b><br>(1.0)        | - 데이터 샘플링 비율<br>- 과적합 제어용                                                                                          |
| <b>feature_fraction</b><br>(1.0) | <b>colsample_bytree</b><br>(1.0) | - 개별트리 학습시 선택되는 피쳐 비율<br>- 과적합 제어용                                                                                 |
| <b>lambda_l2</b><br>(0)          | <b>reg_lambda</b><br>(0)         | - L2 Regularization 적용 값<br>- 피쳐 개수가 많을 때 적용을 검토<br>- 클수록 과적합 감소 효과                                                |
| <b>lambda_l1</b><br>(0)          | <b>reg_alpha</b><br>(0)          | - L1 Regularization 적용 값<br>- 피쳐 개수가 많을 때 적용을 검토<br>- 클수록 과적합 감소 효과                                                |
| <b>objective</b>                 | <b>objective</b>                 | - ‘reg:linear’ : 회귀<br>- binary:logistic : 이진분류<br>- multi:softmax : 다중분류, 클래스 반환<br>- multi:softprob : 다중분류, 확률반환 |

으로, 손실이 큰 부분을 파고들고

하지만, 과적합의 위험이 존재함.

이, 더 복잡한 패턴을 빨리 학습하고 오류를 빠르게

따라서 어디까지 파고들지,  
얼마나 세밀하게 볼지 등의

요소는

**\*\*파라미터 설정\*\***을 통해  
조절해야함.



# RFECV 로 Feature 탐색 과정

- 중요하지 않은 피처를 반복적으로 제거하면서, 교차검증을 통해 최적의 피처 집합을 선택하는 방법

```
# == 1) 베이스라인 LGBM + OOF + Feature Importance ==
base_params = dict(
    objective="regression", metric="rmse",
    learning_rate=0.03, n_estimators=2000,
    num_leaves=63, max_depth=-1, min_child_samples=100,
    subsample=0.8, colsample_bytree=0.8,
    reg_lambda=1.0, reg_alpha=0.0,
    n_jobs=-1, random_state=42
)

# 시계열 분할 (값 간격은 불규칙해도 '순서'만 지키면 OK)
tscv = TimeSeriesSplit(n_splits=5)

oof = np.full(len(X_full), np.nan, dtype="float32")
fi_accum = np.zeros(X_full.shape[1], dtype="float64")

for fold, (tr_idx, va_idx) in enumerate(tscv.split(X_full), 1):
    X_tr, y_tr, w_tr = X_full[tr_idx], y_full[tr_idx], w_full[tr_idx]
    X_va, y_va, w_va = X_full[va_idx], y_full[va_idx], w_full[va_idx]

    model = lgb.LGBMRegressor(**base_params)
    model.fit(
        X_tr, y_tr,
        sample_weight=w_tr,
        eval_set=[(X_va, y_va)],
        eval_sample_weight=w_va,
        eval_metric="rmse",
        callbacks=[lgb.early_stopping(stopping_rounds=200, verbose=False)]
    )

    oof[va_idx] = model.predict(X_va, num_iteration=model.best_iteration_)
    fi_accum += model.booster_.feature_importance(importance_type="gain")

mask = ~np.isnan(oof)
wrmse_base = np.sqrt(np.sum(w_full[mask] * (y_full[mask] - oof[mask])**2) / np.sum(w_full[mask]))
```

partition 6 사용 : 최신성 반영

| [A] Top 10 FI |            |               |
|---------------|------------|---------------|
|               | feature    | importance    |
| 0             | feature_61 | 249471.914262 |
| 1             | feature_21 | 212383.131758 |
| 2             | feature_24 | 174700.155873 |
| 3             | feature_22 | 172481.992024 |
| 4             | feature_20 | 166226.800695 |
| 5             | feature_25 | 156811.061631 |
| 6             | feature_29 | 153942.740649 |
| 7             | feature_06 | 147696.161499 |
| 8             | feature_28 | 135640.246544 |
| 9             | feature_30 | 131416.565393 |



# RFECV 로 Feature 탐색 과정

- 중요하지 않은 피처를 반복적으로 제거하면서, 교차검증을 통해 최적의 피처 집합을 선택하는 방법

```
# == 1) 베이스라인 LGBM + OOF + Feature Importance ==
base_params = dict(
    objective="regression", metric="rmse",
    learning_rate=0.03, n_estimators=2000,
    num_leaves=63, max_depth=-1, min_child_samples=100,
    subsample=0.8, colsample_bytree=0.8,
    reg_lambda=1.0, reg_alpha=0.0,
    n_jobs=-1, random_state=42
)

# 시계열 분할 (값 간격은 불규칙해도 '순서'만 지켜주면 됨)
tscv = TimeSeriesSplit(n_splits=5)

oof = np.full(len(X_full), np.nan, dtype="float32")
fi_accum = np.zeros(X_full.shape[1], dtype="float64")

for fold, (tr_idx, va_idx) in enumerate(tscv.split(X_full), 1):
    X_tr, y_tr, w_tr = X_full[tr_idx], y_full[tr_idx], w_full[tr_idx]
    X_va, y_va, w_va = X_full[va_idx], y_full[va_idx], w_full[va_idx]

    model = lgb.LGBMRegressor(**base_params)
    model.fit(
        X_tr, y_tr,
        sample_weight=w_tr,
        eval_set=[(X_va, y_va)],
        eval_sample_weight=w_va,
        eval_metric="rmse",
        callbacks=[lgb.early_stopping(stopping_rounds=200, verbose=False)]
    )

    oof[va_idx] = model.predict(X_va, num_iteration=model.best_iteration_)
    fi_accum += model.booster_.feature_importance(importance_type="gain")

mask = ~np.isnan(oof)
wrmse_base = np.sqrt(np.sum(w_full[mask] * (y_full[mask] - oof[mask])**2) / np.sum(w_full[mask]))
```

## 유의미한 결과가 도출되지 않음

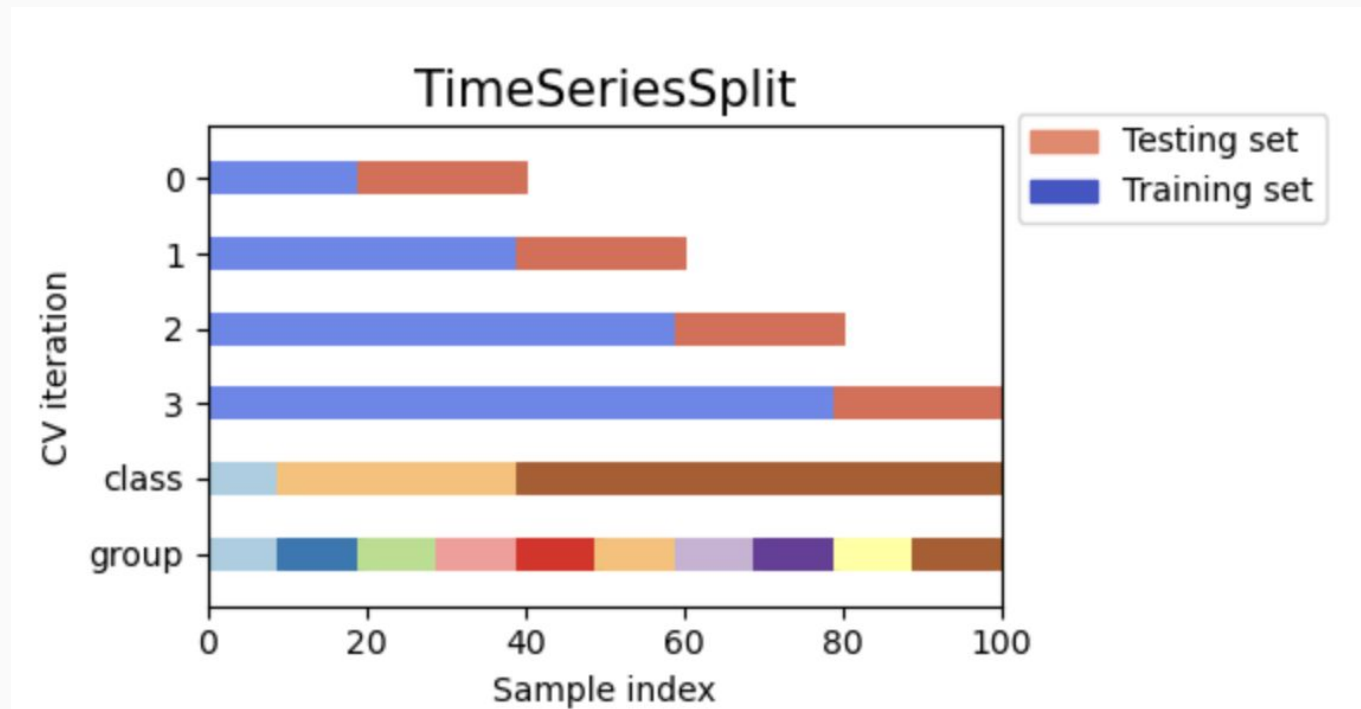
partition 6 사용 : 최신성 반영

[A] Top 10 FI

| rank | feature    | importance    |
|------|------------|---------------|
| 0    | feature_6  | 249701.901775 |
| 1    | feature_21 | 212330.131775 |
| 2    | feature_24 | 174700.155873 |
| 3    | feature_22 | 172481.992024 |
| 4    | feature_20 | 166226.800695 |
| 5    | feature_25 | 156811.061631 |
| 6    | feature_29 | 153942.740649 |
| 7    | feature_06 | 147696.161499 |
| 8    | feature_28 | 135640.246544 |
| 9    | feature_30 | 131416.565393 |

# TimeSeriesSplit 이란?

**k번째까지는 학습, k+1번째 구간은 검증**  
→ 시계열 데이터의 시간 흐름을 보존하는 교차 검증  
방식





# TimeSeriesSplit 이란?

예시)

k번째까지는 학습, k+1번째 구간은 검증

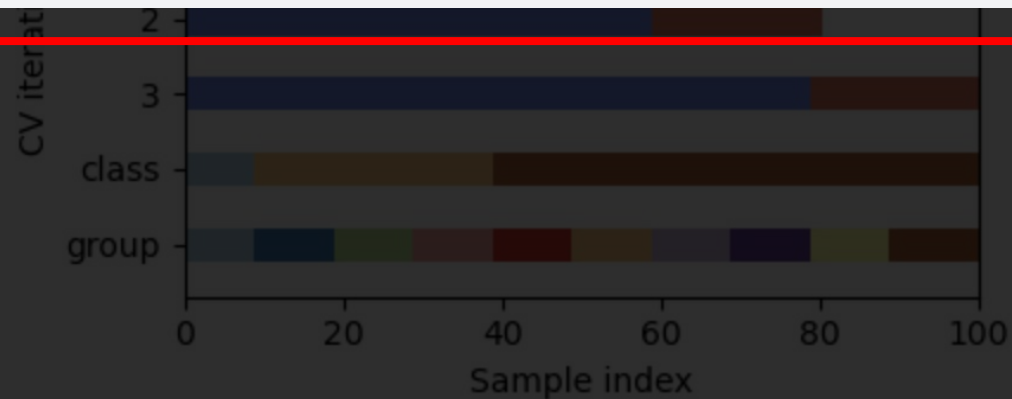
# 샘플 데이터: 인덱스 0~9

`X = np.arange(10)`

# TimeSeriesSplit 예시

`tscv = TimeSeriesSplit(n_splits=5)`

```
[{'Fold': 1, 'Train_idx': [0, 1, 2, 3, 4], 'Valid_idx': [5]}, # 첫 번째 Train 세트는 (전체 샘플 수) ÷ (k+1) 크기부터 시작
{'Fold': 2, 'Train_idx': [0, 1, 2, 3, 4, 5], 'Valid_idx': [6]},
{'Fold': 3, 'Train_idx': [0, 1, 2, 3, 4, 5, 6], 'Valid_idx': [7]},
{'Fold': 4, 'Train_idx': [0, 1, 2, 3, 4, 5, 6, 7], 'Valid_idx': [8]},
{'Fold': 5, 'Train_idx': [0, 1, 2, 3, 4, 5, 6, 7, 8], 'Valid_idx': [9]}]
```



## Train Data 학습 및 Test Data 검증 과정

- Feature importance로 뽑은 피처를 활용해 여러 하이퍼파라미터를 조정하며 Train data (partition 0~6) 학습하고 Test data 로 검증함

### TimeSeriesSplit 5-Fold CV (train=part0~6)

```
- Fold 1: MSE 0.933747 | wMSE 0.845995 | MAE 0.642623 | R^2 0.0062 | best_iter 198
- Fold 2: MSE 0.862891 | wMSE 0.750036 | MAE 0.608194 | R^2 0.0088 | best_iter 457
- Fold 3: MSE 0.910641 | wMSE 0.839873 | MAE 0.595304 | R^2 0.0132 | best_iter 363
- Fold 4: MSE 0.767979 | wMSE 0.699310 | MAE 0.560469 | R^2 0.0066 | best_iter 494
- Fold 5: MSE 0.807908 | wMSE 0.771721 | MAE 0.583596 | R^2 0.0085 | best_iter 625
```

➡ 각 Fold별 성능 측정

### CV 요약

```
MSE      : 0.856633 ± 0.061865
wMSE     : 0.781387 ± 0.055512
MAE      : 0.598037 ± 0.027271
R^2      : 0.0087 ± 0.0025
▶ 평균 best_iteration ≈ 427
```

➡ 평균 성능을 확인하여, 최종 학습 시 사용할 **best\_iteration(최적 학습 수)** 도출

## 평가 지표

### 평균제곱오차

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- 예측값과 실제값 차이(오차)를 제곱해 평균.

### 가중평균제곱오차

$$wMSE = \frac{\sum_{i=1}^n w_i (y_i - \hat{y}_i)^2}{\sum_{i=1}^n w_i}$$

- 중요도가 높은 데이터에 가중치를 곱해 반영한 MSE. (금융 데이터처럼 샘플 중요도가 다를 때 적합)

### 평균절대오차

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

- 예측값과 실제값 차이(오차)의 절댓값 평균.

### 결정계수

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

- 모델이 데이터를 얼마나 잘 설명하는지의 비율. (1에 가까울수록 설명력 높음)

### 피어슨 상관계수

$$Corr = \frac{Cov(y, \hat{y})}{\sigma_y \sigma_{\hat{y}}}$$

- 예측값과 실제값이 같은 방향으로 움직이는지 측정.

### 가중 피어슨 상관계수

$$WCorr = \frac{\sum w_i (y_i - \bar{y}_w)(\hat{y}_i - \bar{\hat{y}}_w)}{\sqrt{\sum w_i (y_i - \bar{y}_w)^2 \cdot \sum w_i (\hat{y}_i - \bar{\hat{y}}_w)^2}}$$

- Corr에 가중치를 반영.

## Train Data 학습 및 Test Data 검증 과정 (1)

- Feature importance로 뽑은 피처를 활용해 여러 하이퍼파라미터를 조정하며 Train data (partition 0~6) 학습하고 Test data 로 검증함

**버전 1)**  
**Partition별 top30 의 교집합**  
(최종 20개 feature)

✓ 최종 Test 성능 (part7~9)

|                     |            |
|---------------------|------------|
| Test MSE            | : 0.697188 |
| Test wMSE           | : 0.657847 |
| Test MAE            | : 0.539481 |
| Test R <sup>2</sup> | : 0.0059   |
| Corr                | : 0.0770   |
| WeightedCorr        | : 0.0772   |

➡ R<sup>2</sup>, Corr이 낮아 예측력이 제한적임

**버전 2)**  
**List 3개 이상 등장하는 피처  
집합**  
(최종 33개 feature)

✓ 최종 Test 성능 (part7~9)

|                     |            |
|---------------------|------------|
| Test MSE            | : 0.694831 |
| Test wMSE           | : 0.655404 |
| Test MAE            | : 0.538704 |
| Test R <sup>2</sup> | : 0.0092   |
| Corr                | : 0.0965   |
| WeightedCorr        | : 0.0984   |

➡ 전반적인 성능이 개선됨.  
feature 갯수의 증가로, 더 일반화된 성능을 확보함.

## Train Data 학습 및 Test Data 검증 과정 (1)

- Feature importance로 뽑은 피처를 활용해 여러 하이퍼파라미터를 조정하며 Train data (partition 0~6) 학습하고 Test data 로 검증함

버전 3) Lag +  
Partition별 top30 의 교집합  
(최종 20개 feature)

✓ 최종 Test 성능 (part7~9)

|                     |            |
|---------------------|------------|
| Test MSE            | : 0.697151 |
| Test wMSE           | : 0.657806 |
| Test MAE            | : 0.539475 |
| Test R <sup>2</sup> | : 0.0059   |
| Corr                | : 0.0773   |
| WeightedCorr        | : 0.0775   |

버전 4) Lag +  
List 3개 이상 등장하는 피처  
집합  
(최종 33개 feature)

✓ 최종 Test 성능 (part7~9)

|                     |            |
|---------------------|------------|
| Test MSE            | : 0.694812 |
| Test wMSE           | : 0.655399 |
| Test MAE            | : 0.538666 |
| Test R <sup>2</sup> | : 0.0093   |
| Corr                | : 0.0967   |
| WeightedCorr        | : 0.0985   |

➡ 성능 지표 전반에서 소폭 개선  
버전 4의 WeightedCorr이 최고치를 기록

## Train Data 학습 및 Test Data 검증 과정 (2)

- Symbol별로 데이터 분할, 예시로 3개 사용
- Feature importance로 뽑은 피처를 활용해 여러 하이퍼파라미터를 조정하며 Train data (partition 0~6) 학습하고 Test data 로 검증함

**버전 5) Symbol별 데이터 분할, 목표 데이터와 상관관계 높은 feature로 예측 수행**  
 (최종 30개 feature)

### ✓ 최종 Test 성능 (part7~9)

Test MSE : 0.497059  
 Test wMSE : 0.503002  
 Test MAE : 0.465162  
 Test R<sup>2</sup> : -0.0000  
 Corr : 0.0514  
 WeightedCorr: 0.0506

Symbol\_id = 0

### ✓ 최종 Test 성능 (part7~9)

Test MSE : 0.561522  
 Test wMSE : 0.549801  
 Test MAE : 0.477339  
 Test R<sup>2</sup> : 0.0040  
 Corr : 0.0693  
 WeightedCorr: 0.0711

Symbol\_id = 1

### ✓ 최종 Test 성능 (part7~9)

Test MSE : 0.761571  
 Test wMSE : 0.792945  
 Test MAE : 0.576877  
 Test R<sup>2</sup> : 0.0019  
 Corr : 0.0438  
 WeightedCorr: 0.0469

Symbol\_id = 2

➡ Symbol 0, 1: 성능 지표가 전반적으로 개선됨  
 Symbol 2: 성능 지표가 전반적으로 저조함

# 결과 해석 및 인사이트

- 중요 피처 선정 방식과 Lag 적용에 따른 성능 변화를 비교하여 인사이트 도출

## 1. Partition별 Top30 교집합 vs 다중 등장 피처 집합

- **Version 1 (Partition별 Top30 교집합)**
  - 안정적으로 주요 피처를 반영했으나, 성능 지표는 제한적 개선
  - Corr, Weighted Corr이 다소 낮아 시장 반응 신호 포착력이 약함
- **Version 2 (List 3회 이상 등장 피처 집합)**
  - 성능이 전반적으로 향상 (특히 WeightedCorr ↑)
  - 반복적으로 중요도가 높은 피처들을 선별 → 일관된 정보가 예측에 기여

## 2. Lag Feature 추가 효과

- **Version 3 (Lag + Partition별 Top30 교집합)**
  - 성능 변화는 크지 않음 → 단순히 lag 추가하는 것만으로는 한계
  - 일부 시계열 패턴 반영에는 기여했으나, noise도 포함된 것으로 보임
- **Version 4 (Lag + List 3회 이상 등장 피처 집합)**
  - 성능 지표 전반에서 가장 개선 → 특히 WeightedCorr 0.0985로 최고치 기록
  - 반복적으로 중요도가 높았던 피처를 기반으로 안정적인 신호 확보
  - Lag 피처 결합으로 시계열 패턴까지 반영
  - 결과적으로, “안정된 피처 선별 + 시간 지연 신호 결합”이 최적임을 확인



## 3. 종합 인사이트

- 단일 partition 기준 피처보다는 **여러 partition에서 공통적으로 중요한 피처**를 모으는 것이 더 효과적
- Lag 피처는 단독 효과는 크지 않지만, **안정적인 피처 선택과 함께 쓸 때 성능 향상**에 기여
- 최적 조합: **“다중 등장 피처 + Lag”** → 실제 시장 예측에 가장 유용
- **Version 5 (Symbol별로 구분하여 예측 수행)**
  - 상관관계수값 불안정
  - 일부 Symbol에서만 성능 개선 → 기존 방법의 개선에 활용



## 한계 및 도전 과제

- 익명 데이터와 모델의 제약으로 인해 해석 · 일반화 · 실시간 적용에 한계 존재

### 1. 피처 이름 익명 처리 -> 해석 불가능

### 2. 데이터 드리프트 가능성

### 3. 적용해보지 못했지만, 실시간 트레이딩 적용 시 **Latency** 문제

### 4. 모델 해석 가능성 부족

- LightGBM 자체는 성능은 좋지만, “왜 이 예측이 나왔는지” 설명이 어려움
- SHAP 등 해석 기법 사용 필요

### 5. 과적합(Overfitting) 위험

- 특정 partition에만 맞춰 학습하면, 미래 환경 변화에 취약
- 일반화 성능 확보가 과제

### 6. Lag Feature 한계

- 시계열 구조를 더 잘 반영할 수 있는 rolling window, trend/momentum 피처 필요

## 요약 및 결론

- 중요 피처 선별과 시계열 신호 반영을 통해 예측 성능과 적용 가능성을 확인

### 1. 핵심 결과

- 피처 전처리 + 중요도 분석  
-> 성능 향상 확인
- Lag 데이터  
-> 시계열 패턴 학습 기여

### 2. 의의

- 익명 피처 환경에서도 의미 있는 분석 가능성 확인
- 실시간 예측 모델링의 기초 확보

### 3. 향후 연구 방향

- 피처 엔지니어링 강화
  - Lag + Rolling Window +  $\Delta$  변화율
- 다양한 모델 실험
  - 딥러닝 기반 시계열 모델 (LSTM, Transformer), 앙상블 기법
- 실시간 환경 검증
  - 스트리밍 데이터 기반 실험
  - 실제 거래 시스템 Latency 고려
- Feature Selection 해석 강화
  - SHAP, Permutation Importance 등 해석 기법 활용